

Nama: Andien Oktriahmah Syarifah

NIM: 245150400111046

Mata Kuliah: Pengembangan Aplikasi Mobile

Kelas: F

MODUL 4 - State Hoisting, Perubahan Konfigurasi, dan Side-effect

State Hosting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
 - a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Angka yang ditampilkan pada antarmuka layar bertambah 1 setiap kali tombol diklik.
 - b. Apa yang menyebabkan hal tersebut terjadi?

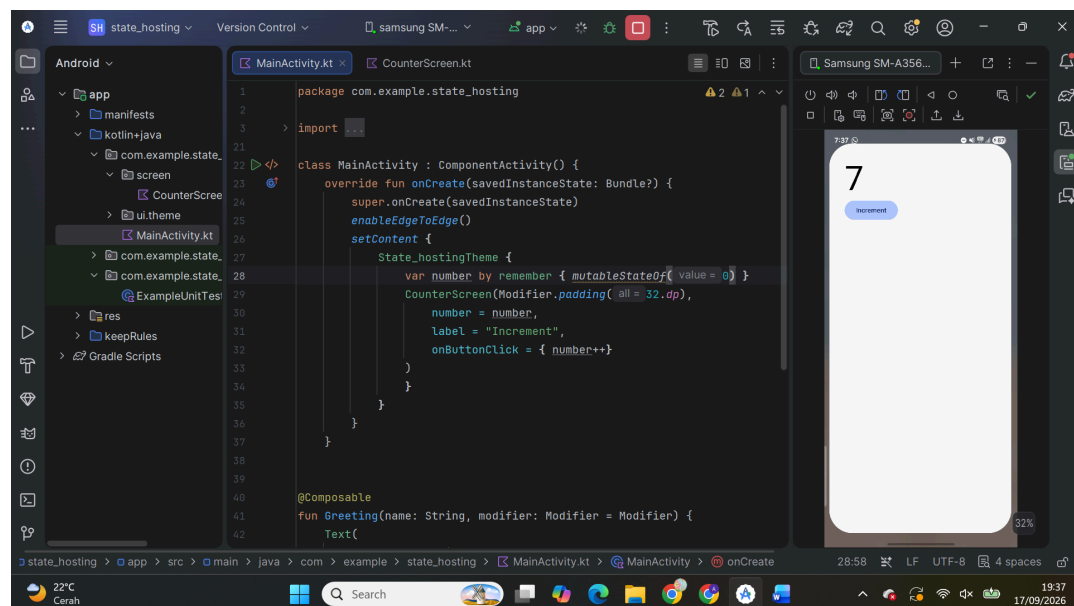
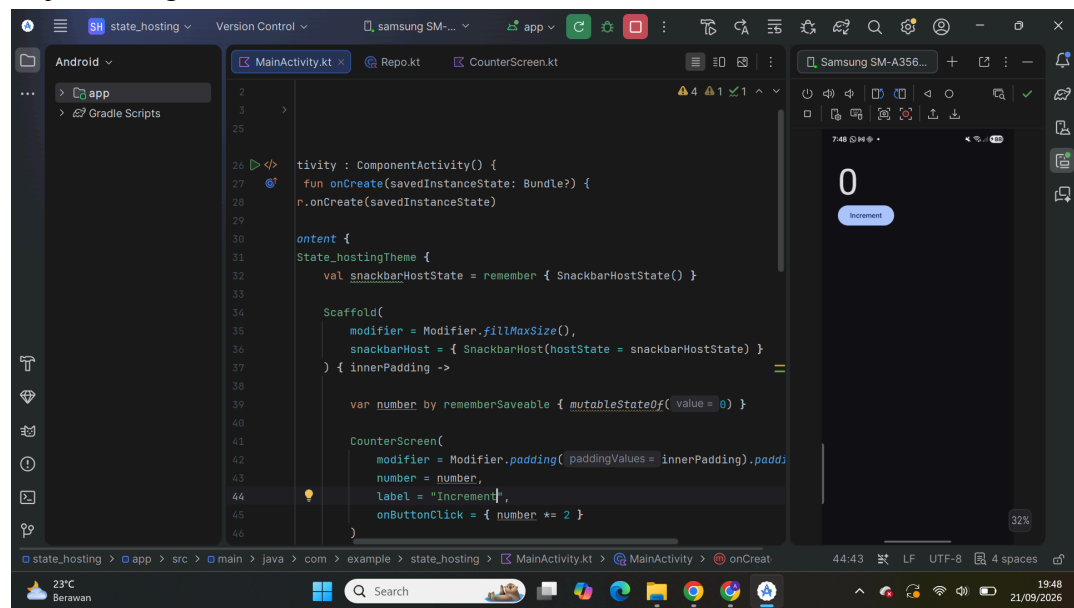
Event `onClick` pada komponen `Button` terpicu saat tombol diklik. Event ini menjalankan lambda `{ number++ }`, yang secara langsung memperbarui nilai variabel *state* `number`. Perubahan nilai *state* ini memberitahu `Jetpack Compose` untuk melakukan rekomposisi (menggambar ulang) bagian antarmuka yang membaca *state* tersebut, sehingga komponen `Text` menampilkan angka yang baru.
 - c. Variabel *state* apa yang berperan pada `Composable Text` yang menampilkan perubahan angka pada `Text` tersebut?

Variabel *state* `number`.
 - d. Variabel *state* apa yang berperan untuk menampilkan label teks pada `Composable Button` yang bertuliskan "Increment" tersebut?

Variabel *state* `label`.
2. Pada percobaan tahap sebelumnya, ubah kode program pada `Composable CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
 - a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan dilakukan secara lokal di dalam fungsi `Composable CounterScreen`:
Ubah nilai inisialisasi *state* `label`: `val label by remember { mutableStateOf("Increment 2") }`
Ubah logika aritmatika di dalam event `onClick`: `Button(onClick = { number += 2 })`

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



3. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen. Ubah signature kode program pada bagian fungsi Composable CounterScreen dari yang sebelumnya:
- ```
fun CounterScreen(modifier: Modifier)
```
- menjadi:
- ```
fun CounterScreen(modifier: Modifier,  
                  number: Int, label: String, onClick: () -> Unit)
```

Ubah body di dalam Composable CounterScreen menjadi:

```
// var number by remember { mutableStateOf(0) }  
// val label by remember { mutableStateOf("Increment") }  
Column(modifier) {  
    Text(text = "$number", fontSize = 72.sp)  
    Button(onClick = onButtonClick) {  
        Text(text = "$label")  
    }  
}
```

Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }  
CounterScreen(Modifier.padding(32.dp),  
    number = number,  
    label = "Increment",  
    onButtonClick = { number++ }  
)
```

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, program berjalan dengan baik dan menampilkan perilaku visual yang sama persis seperti sebelumnya (angka bertambah 1 setiap diklik).
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment dilakukan di dalam fungsi onCreate(), tepatnya pada argumen lambda onButtonClick = { number++ } saat pemanggilan fungsi CounterScreen.
 - c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State dideklarasikan di dalam fungsi onCreate(), sebelum blok pemanggilan CounterScreen.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

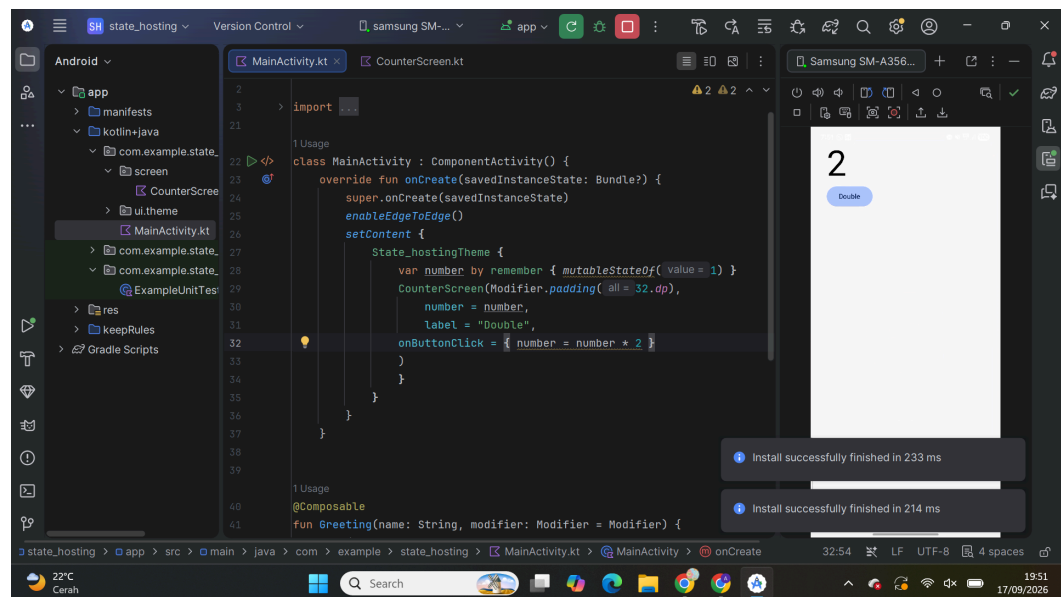
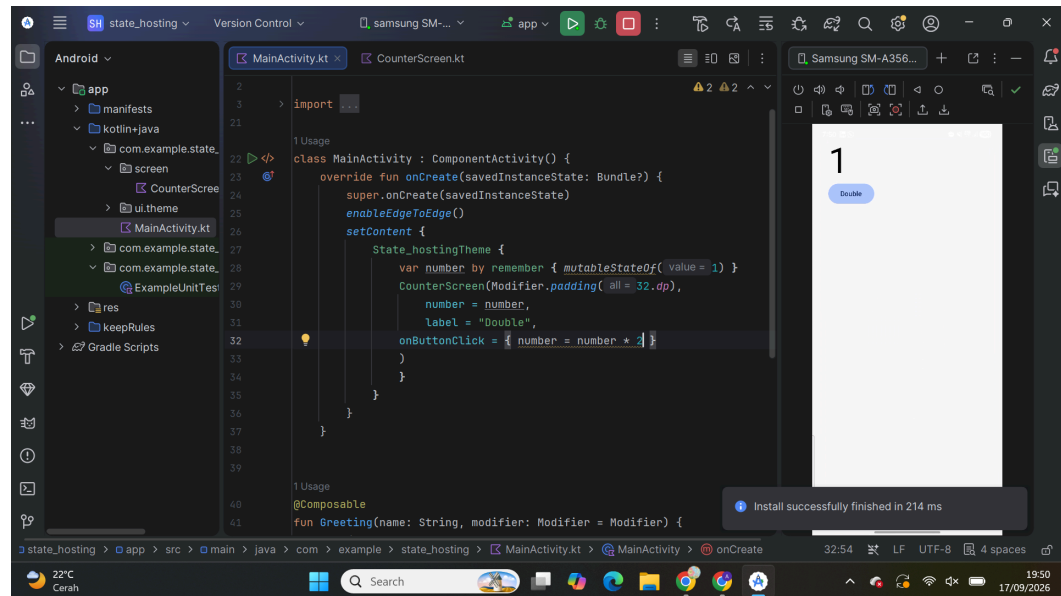
- a. Label tombol "Increment" menjadi "Double"
- b. Nilai awal variabel number dari 0 menjadi 1.
- c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan `Composable CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi `Composable CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan. (Keterangan: Merujuk pada langkah ke-5 di atas).

```
var number by remember { mutableStateOf(1) } // Nilai awal diubah  
menjadi 1
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double", // Label diubah menjadi "Double"  
    onClick = { number *= 2 } // Logika increment diganti menjadi  
    penggandaan  
)
```

6. Jalankan program

- a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Semua perubahan dilakukan di dalam fungsi onCreate() yang bertindak sebagai Composable (pemanggil).

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

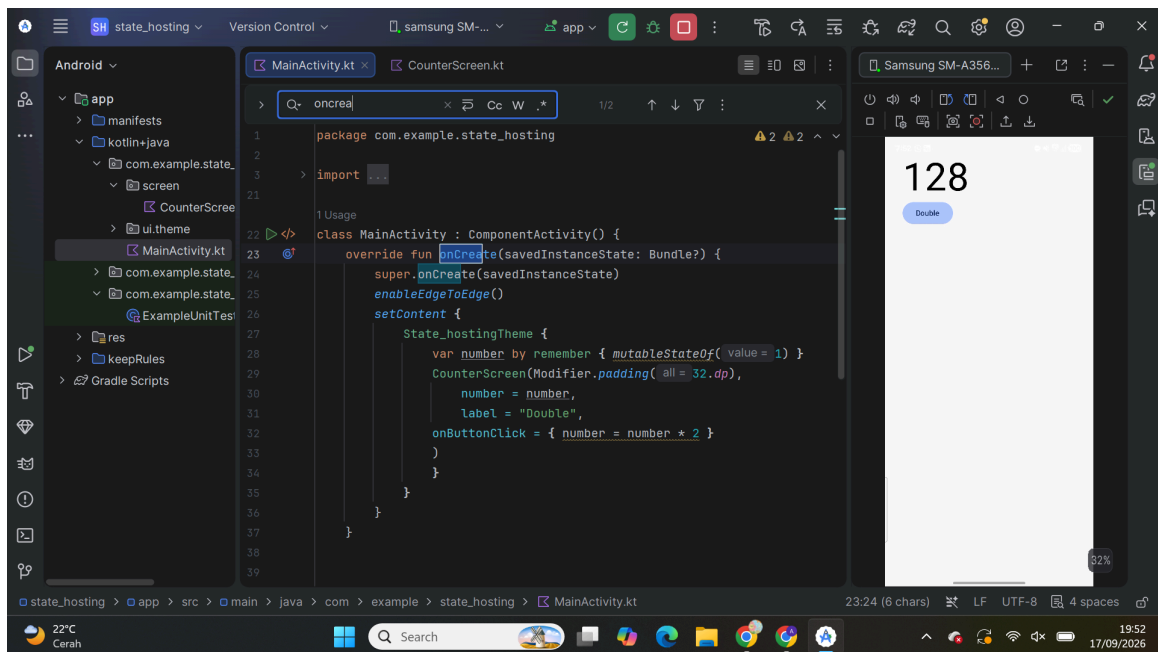
Tidak ada perubahan sama sekali di dalam blok fungsi CounterScreen. Komponen tersebut murni hanya menerima data (*state*) dan meneruskan interaksi (*event*) ke luar.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

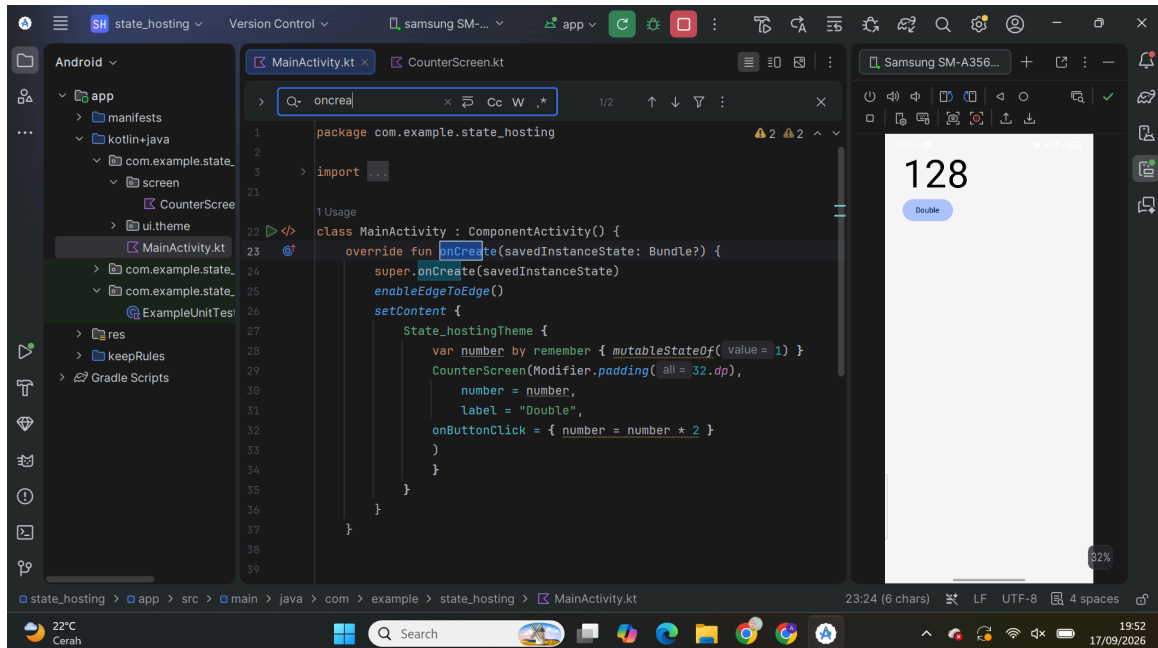
State hoisting membuat komponen antarmuka (Composable anak) menjadi *stateless* (tidak menyimpan data sendiri) sehingga jauh lebih fleksibel dan *reusable* (bisa digunakan kembali). Ketika ada perubahan logika bisnis (seperti mengubah increment menjadi fungsi perkalian) atau perubahan nilai awal, kita tidak perlu membongkar struktur kode elemen antarmuka tersebut, melainkan hanya perlu mengubah logika yang dikirimkan dari fungsi induknya.

Perubahan Konfigurasi

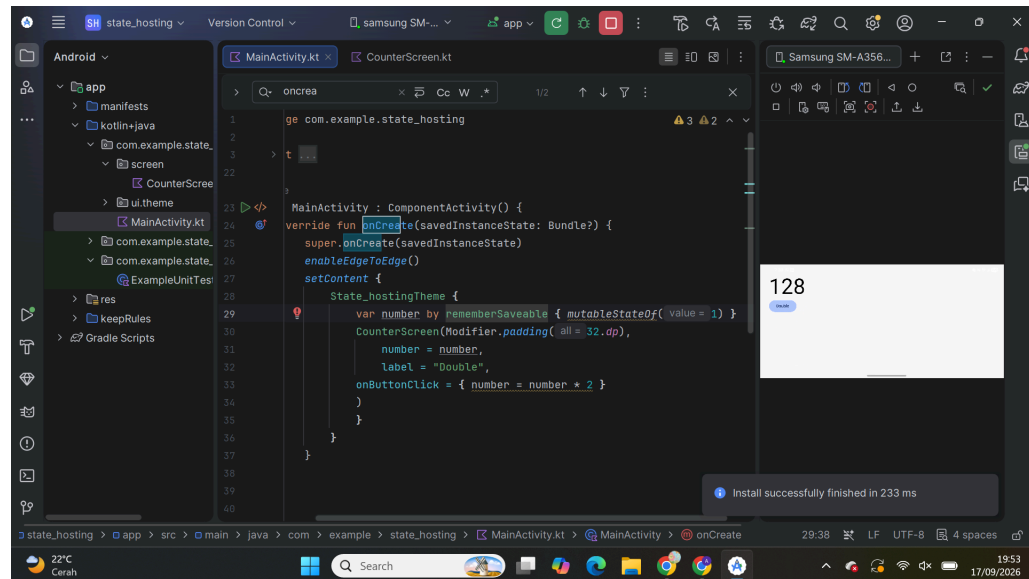
1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian? Lakukan import dependency atas perubahan konstruk remember menjadi rememberSaveable yang dilakukan (dengan memposisikan kursor pada konstruk rememberSaveable tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela editor Android Studio). Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.
 3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan adalah **nilai terakhir sebelum layar diputar** 128. Hal ini terjadi karena `rememberSaveable` secara otomatis mengemas dan menyimpan nilai *state* ke dalam objek `Bundle` milik Android. Data di dalam `Bundle` kebal terhadap proses siklus daur ulang `Activity` akibat perubahan konfigurasi, sehingga nilai sebelumnya berhasil dipulihkan dengan utuh.

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

`remember` digunakan untuk mencegah nilai *state* terreset saat antarmuka digambar ulang (rekomposisi). Sedangkan `rememberSaveable` melakukan perlindungan ganda: melindungi nilai dari efek rekomendasi dan menyimpannya agar tidak hilang saat sistem operasi Android menghancurkan dan membuat ulang proses aplikasi akibat perubahan konfigurasi sistem (rotasi, ubah ukuran jendela, atau transisi *dark mode*).

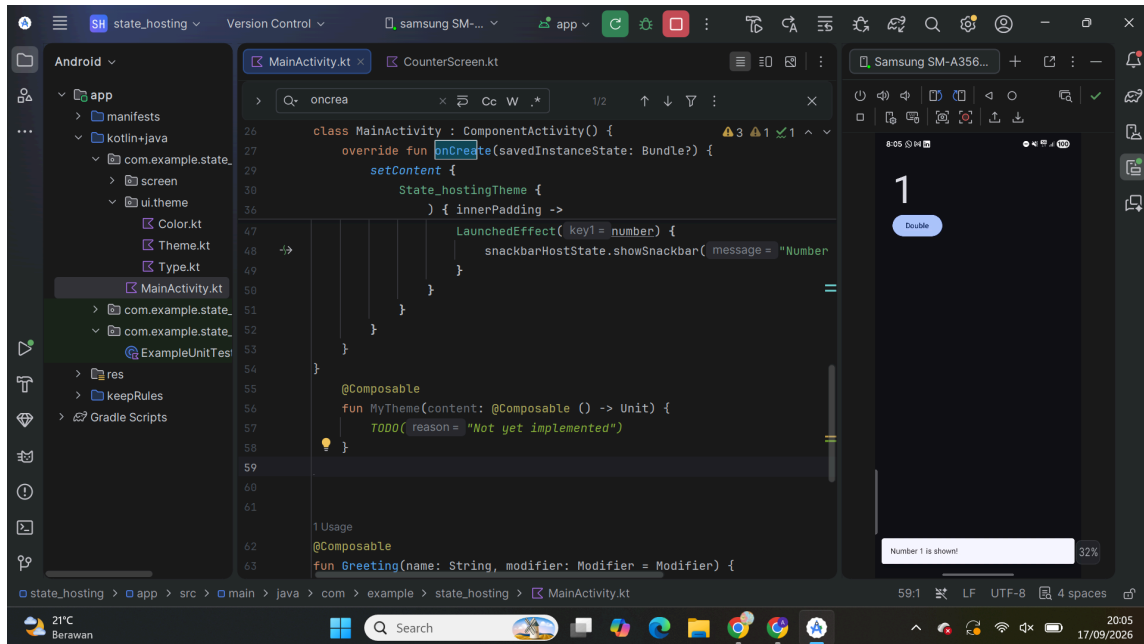
4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh kasus nyatanya adalah mempertahankan teks ulasan yang sedang diketik pengguna di dalam sebuah *Form/TextField*, atau menjaga posisi urutan pergeseran (*scroll position*) pada layar daftar keranjang belanja. Manfaatnya adalah mencegah frustrasi pengguna; jika mereka tidak sengaja merotasi layar gawai mereka, teks panjang yang sudah diketik tidak akan hilang secara tiba-tiba.

Side Effect

Aktivitas Percobaan 1

1. Jalankan program,



- Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?

Layar menampilkan angka 1, dan muncul *SnackBar* (notifikasi kecil di bawah) bertuliskan "Number 1 is shown!".

- Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

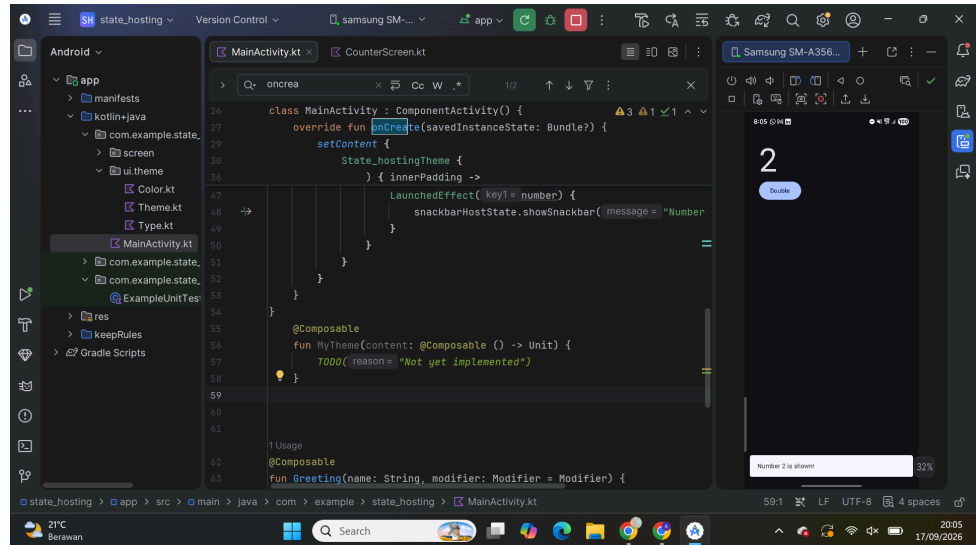
Blok fungsi *LaunchedEffect* memang dirancang untuk dieksekusi secara otomatis saat pertama kali komponen tersebut memasuki siklus hidup antarmuka layar (masuk ke dalam *Composition*).

- Apa yang memicu pesan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?

Pesan tersebut dipicu oleh proses evaluasi awal *Side-effect*. Berada di blok kode:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!") }
```

2. Klik button "Double" satu kali,

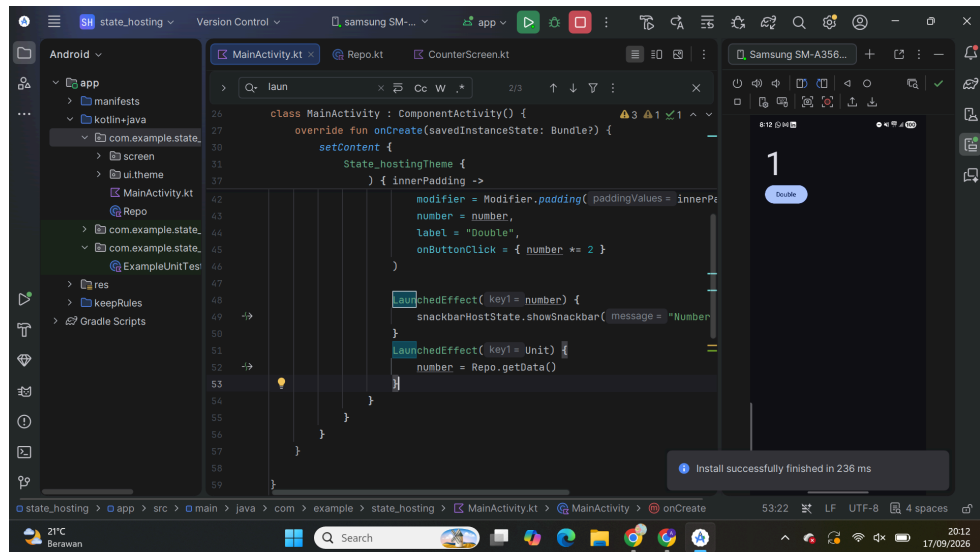


- a. Apa yang terjadi?
Angka di tengah layar menjadi 2, dan beberapa saat kemudian akan muncul *SnackBar* baru dengan pesan "Number 2 is shown!".
- b. Mengapa hal tersebut terjadi? Jelaskan!
Nilai dari parameter *key* yang diberikan kepada *LaunchedEffect* (yakni variabel *number*) telah berubah. Sistem membaca adanya perbedaan antara *key* lama (1) dan *key* baru (2). Karena terdeteksi perubahan, operasi *coroutine* lama dibatalkan dan blok operasi yang sama langsung dieksekusi ulang dengan menyuntikkan nilai *state* yang baru.

Aktivitas Percobaan 2

1. Jalankan program.

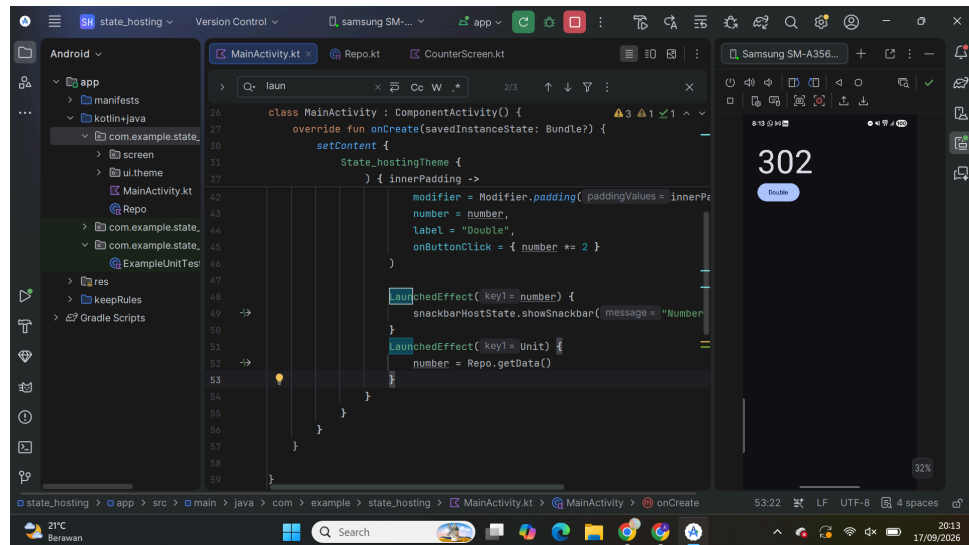
- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



Layar akan menampilkan angka 1.

- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Angka di tengah layar akan berubah secara otomatis menjadi angka acak (karena eksekusi fungsi *randomize*), dan *SnackBar* menampilkan pesan sesuai dengan angka acak tersebut.



- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan tersebut dipicu oleh selesainya durasi penundaan instruksi asinkron `delay(2000)` di latar belakang. Kodenya berada di:
`LaunchedEffect(Unit) { number = Repo.getRandom() }`

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)` bersifat dinamis dan reaktif, di mana eksekusi perintahnya terkunci pada perubahan variabel state spesifik. Sedangkan `LaunchedEffect(Unit)` bersifat statis karena ia bergantung pada nilai `Unit` (sebuah nilai tetap di Kotlin yang tidak pernah berubah), sehingga efek di dalamnya kebal terhadap segala bentuk perubahan data antarmuka.

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- `LaunchedEffect(Unit)`: Hanya dijalankan **tepat satu kali** seumur hidup siklus antarmuka, yakni pada detik pertama kali layar dimuat.
- `LaunchedEffect(state)`: Dijalankan pada saat pertama kali layar dimuat, **DAN** akan dieksekusi ulang setiap detik variabel state-nya berganti nilai.